

Tesla Routing: Non-Linear Payment Routing in the Bitcoin Lightning Network

Raehat Singh Nanda

raehatsinghnanda@gmail.com

Abstract

The Bitcoin Lightning Network relies on source-based onion routing to forward payments through a sequence of intermediary nodes. Current routing paths are strictly linear: each forwarding node may pass the payment to exactly one successor. When a node cannot forward a payment — due to insufficient local channel balance or temporary unavailability — the entire payment fails and must be retried. This paper proposes Tesla Routing, a protocol extension that allows forwarding nodes to select from multiple pre-encoded alternative successor nodes within a single onion packet. By distributing branching information across a fixed-size 1,300-byte SPHINX onion payload using a novel split-and-interleave construction, Tesla Routing increases payment success probability without compromising the privacy guarantees of SPHINX-based onion routing and without fundamental changes to the existing Lightning Network protocol stack. We describe the onion construction algorithm, the fee equalization mechanism, versioning strategy, and open problems for future work.

I. INTRODUCTION

The Bitcoin Lightning Network [1] is a layer-2 payment protocol enabling fast, low-cost transactions by routing payments through pre-funded, bidirectional payment channels. Rather than recording each transaction on the Bitcoin blockchain, Lightning nodes lock funds in channels and update balances off-chain, settling to the blockchain only when channels close.

Routing a payment from source to destination requires finding a path of connected channels through the network. This path-finding problem is complicated by a fundamental privacy constraint: nodes do not broadcast their current channel balances. Revealing balance information would allow third parties to infer transaction volumes and monitor participants' financial activity. As a result, a source node selects a path with incomplete information and discovers failures only after the payment is in flight.

Current Lightning routing is strictly linear. Once a path $A \rightarrow B \rightarrow C \rightarrow D$ is selected and forwarded, each intermediary node passes the payment to exactly one successor. If node C has insufficient outbound balance,

the payment fails and must be retried along a different path — adding latency, increasing computational cost, and degrading user experience for time-sensitive payments.

This paper proposes Tesla Routing, named for the branching, forking shape of a lightning bolt — a visual analogy for the non-linear paths the protocol enables. The core idea is to encode multiple alternative forwarding paths within a single onion packet, allowing a node to try a backup successor if its primary fails. We show how this can be achieved within the existing fixed-size 1,300-byte SPHINX payload, and describe the required changes to the Lightning Network protocol.

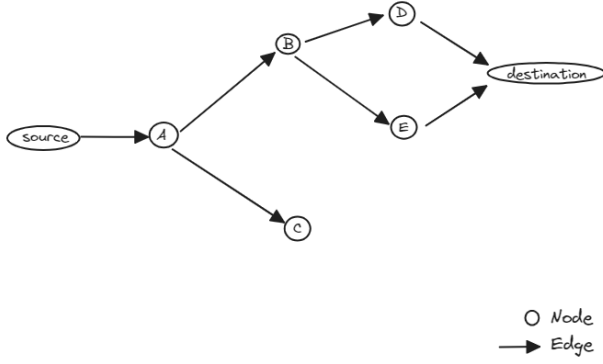


Fig. 1. Example Lightning Network topology illustrating multiple paths from source to destination.

II. BACKGROUND

A. Lightning Network Routing

Routing in the Lightning Network is a two-phase process: path-finding and payment forwarding. In path-finding, the source selects a sequence of nodes and channels using a gossip-based network map. Nodes broadcast channel announcements and updates but not current balances. The source uses probabilistic models — based on channel capacity, historical success rates, and heuristics — to estimate path success likelihood [2].

When a payment fails mid-route, the failing node generates an encrypted error that propagates back to the source. The source then selects a new path, avoiding the failed edge, and retries. This backtracking can require multiple round-trips, significantly increasing payment latency.

B. SPHINX Onion Routing

Payment forwarding in the Lightning Network uses SPHINX [3], a cryptographic packet format providing source routing with the following privacy properties: (i) each intermediary node learns only the identity of its immediate predecessor and successor; (ii) no node learns the total path length or its position within the path; (iii) the destination is not revealed to intermediary nodes.

The onion packet structure is shown in Fig. 2. The 1,300-byte onion payload contains hop payloads in Type-Length-Value (TLV) format per BOLT 4 [4], each specifying: `short_channel_id` (where to forward), `amt_to_forward` (amount in millisatoshis), and `outgoing_cltv_value` (timelock for the outgoing HTLC).

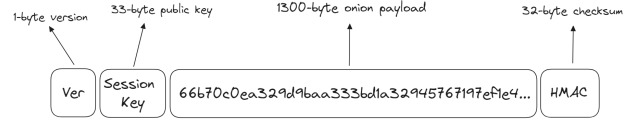


Fig. 2. SPHINX onion packet format: 1-byte version, 33-byte session key, 1,300-byte payload, 32-byte HMAC.

Packet construction proceeds backwards from the destination. For each hop, the source prepends that hop's payload to the payload buffer, shifts existing data right, trims to 1,300 bytes, and XORs with a pseudo-random byte stream (ChaCha20) derived from a rho key computed from the shared secret between the source and that hop's node. Each node, upon receipt, reverses this process to recover its payload and passes the remainder forward.

C. Hash Time-Locked Contracts

Payment atomicity is enforced using Hash Time-Locked Contracts (HTLCs). The source generates a random preimage r and its hash $H(r)$. Each node locks funds conditional on the revelation of r . When the destination reveals r , the preimage propagates back and each node settles its HTLC. If the payment fails, HTLCs expire after a timelock and funds are returned. Tesla Routing must be compatible with HTLC atomicity, as addressed in Section V.

III. MOTIVATION

Consider the topology in Fig. 1. Source A wishes to send a payment to destination F. A selects path $A \rightarrow B \rightarrow D \rightarrow F$ and forwards an onion. Node D, however, has insufficient outbound balance and returns an error. A must now retry via $A \rightarrow B \rightarrow C \rightarrow E \rightarrow F$, constructing a new onion and restarting the payment entirely.

This retry cycle has compounding costs:

- Latency: each round-trip from source to failing node and back adds hundreds of milliseconds.
- HTLC lock-up: funds locked in failed HTLCs remain unavailable until timelocks expire.
- Computational overhead: a new onion must be constructed with new shared secrets.
- User experience degradation: point-of-sale payments may time out before retries succeed.

If the source could encode both $A \rightarrow B \rightarrow D \rightarrow F$ and $A \rightarrow B \rightarrow C \rightarrow E \rightarrow F$ in a single onion, node B could

try D first and fall back to C without any communication back to the source. This is the key insight behind Tesla Routing.

IV. TESLA ROUTING DESIGN

A. Overview

Tesla Routing extends SPHINX to allow a node to encode two or more alternative successor nodes within a single hop payload. When a forwarding node encounters a payload listing multiple outgoing channels, it attempts them in order — no source communication required. The design is constrained to: (i) keep the onion payload exactly 1,300 bytes; (ii) preserve all SPHINX privacy properties; (iii) remain backward-compatible via version negotiation.

B. Protocol Version Negotiation

The current onion version byte is set to 1. Tesla Routing introduces version 2. A node receiving a version-2 onion without non-linear routing support must reject it. To avoid this, nodes include their maximum supported onion version in node_announcement gossip messages [5]. Source nodes consult the gossip map and construct version-2 onions only for paths composed entirely of version-2-capable nodes. Version-1 paths continue to function unchanged.

C. Fee Equalization

Each hop payload carries a single `amt_to_forward` field. In linear routing this is deterministic; in non-linear routing, different alternative paths may have different total fees. For example, if each node charges 10,000 msat, path $A \rightarrow B \rightarrow C \rightarrow E \rightarrow F$ (three downstream hops) requires B to forward 10,200,000 msat, while $A \rightarrow B \rightarrow D \rightarrow F$ (two hops) requires only 10,100,000 msat.

Since a node can carry only one `amt_to_forward`, Tesla Routing sets it to the maximum across all alternative paths. Let $P_1, P_2, \dots, P_k$ be the k alternative paths from branching node B to destination F, and let $\text{fee}(P_i)$ denote the total fee on path P_i . Then:

$$\text{amt_to_forward}(B) = \text{amount} + \max\{\text{fee}(P_1), \dots, \text{fee}(P_k)\}$$

Nodes on cheaper paths receive an overpayment equal to the difference from the maximum-fee path. This overcompensation rewards nodes for their availability

as fallback routes and preserves the single-field constraint.

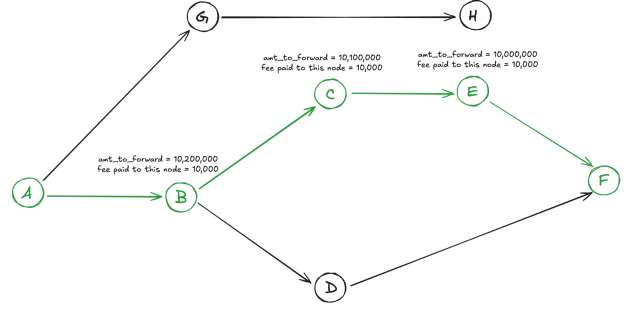


Fig. 3. Fee structure for path $A \rightarrow B \rightarrow C \rightarrow E \rightarrow F$. `amt_to_forward` and fee paid to each node are labeled.

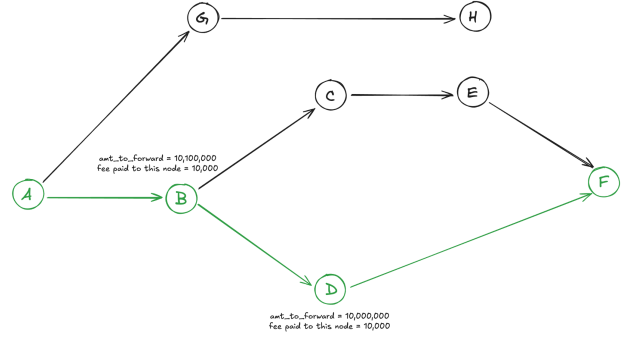


Fig. 4. Fee structure for path $A \rightarrow B \rightarrow D \rightarrow F$. Node D receives extra compensation when both paths are encoded.

D. Onion Construction: Two Alternative Paths

We describe onion construction for the two-path case: $A \rightarrow B \rightarrow \{C \rightarrow E, D\} \rightarrow F$, where B has alternative successors C and D, both reaching destination F (Fig. 5).

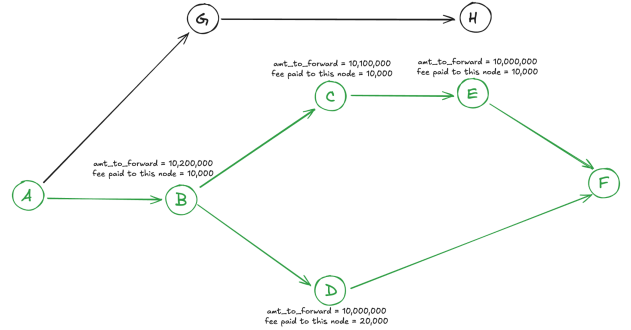


Fig. 5. Combined non-linear topology encoding both paths $A \rightarrow B \rightarrow C \rightarrow E \rightarrow F$ and $A \rightarrow B \rightarrow D \rightarrow F$.

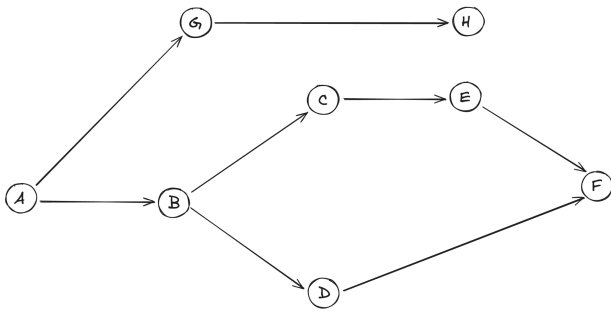


Fig. 6. *Alternative linear topology $A \rightarrow B \rightarrow D \rightarrow F$ for reference.*

1) *Destination Payload (f-onion):*

Construction begins at destination F. F's hop payload (Fig. 7) is prepended to 1,300 bytes of zero filler, trimmed to 1,300 bytes, and XORed with the rho-key stream derived from the A–F shared secret. This produces the f-onion. Two identical copies are required since F terminates both alternative paths.

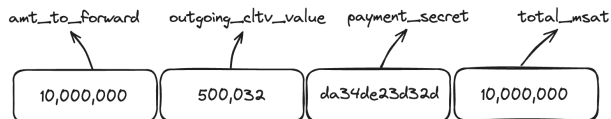


Fig. 7. Destination node F 's hop payload: `amt_to_forward`, outgoing `cltv` value, payment secret, total `msat`.

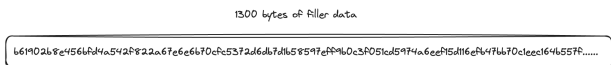


Fig. 8. 1,300-byte filler data appended during f -onion construction.

2) *Single-Path Hops (e-onion, d-onion)*:

For nodes lying on only one alternative path — node E (on path $C \rightarrow E \rightarrow F$) and node D (on path $D \rightarrow F$) — onion construction is identical to standard SPHINX. Each node's hop payload is prepended to its successor's onion, trimmed to 1,300 bytes, and XORed with the appropriate rho-key stream to produce the e-onion and d-onion respectively. The payloads for nodes E and D are shown in Fig. 9.

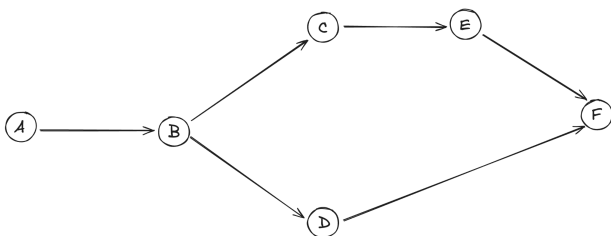


Fig. 9. *Non-linear routing topology showing node B branching to both $C \rightarrow E$ and D, converging at F.*

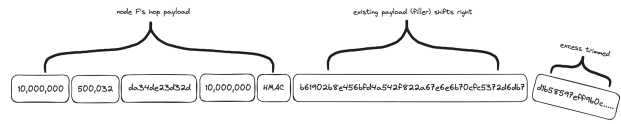


Fig. 10. Hop payloads for nodes E and D:
short_channel_id, amt_to_forward, outgoing_cltv_value.

3) *c*-onion Construction:

Node C lies on only one path ($C \rightarrow E \rightarrow F$) and is constructed by standard SPHINX onion wrapping, taking node E's payload and wrapping it with C's shared secret. This produces the c-onion (Fig. 11).

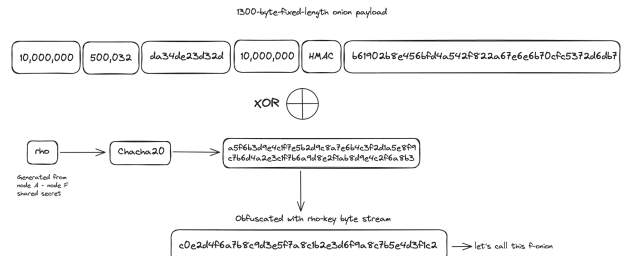


Fig. 11. *e*-onion construction: node *E*'s hop payload is prepended and XORed with the *A*–*E* rho-key stream.

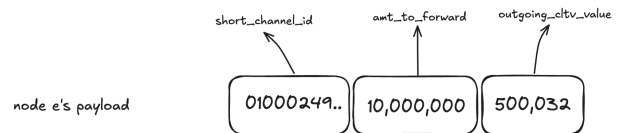
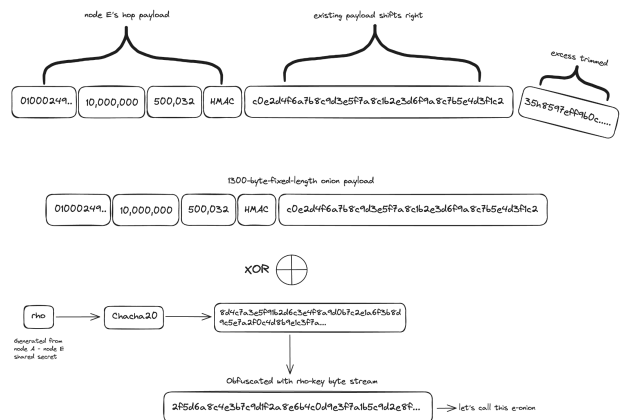


Fig. 12. *d*-onion construction: node *D*'s hop payload is prepended and XORed with the *A*–*D* rho-key stream.

4) *Branching Node B (b-onion)*:

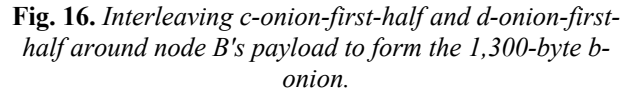
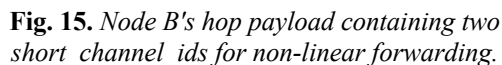
Node B's hop payload (Fig. 13) lists two `short_channel_id` values — one for channel B–C and one for B–D — along with the equalized `amt` to forward and outgoing `cltv` value.



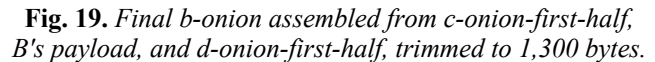
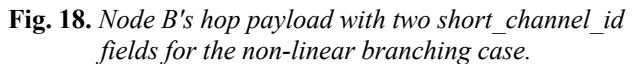
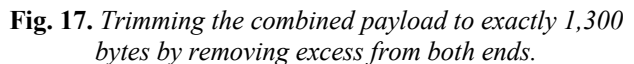
The b-ion is constructed using the following split-and-interleave procedure:

-
- node D's top payload
- existing payload sits right
- excess trimmed
- 1300-byte fixed-length onion payload
- XOR
- obfuscated with rkey byte stream
- let's call this onion

Node B, upon receiving and decrypting its layer, finds two `short_channel_ids`. It interprets the first half of the remaining payload as the c-onion to forward to C, and the second half as the d-onion to forward to D. It first attempts C; on failure, it attempts D.



For k alternative successors, each successor's onion contributes its first $1/k$ of its payload (instead of one-half). Node B's payload is placed centrally, flanked by the first- $1/k$ portions. For $k = 4$, each contributes 325 bytes; for $k = 3$, approximately 433 bytes each (with minor padding to reach exactly 1,300 bytes). As k increases, less information from each successor's onion survives, creating a trade-off between branching factor and onion fidelity. In practice, $k = 2$ is the most useful and introduces the least information loss.



Tesla Routing preserves the core privacy properties of SPHINX. Each node derives its per-hop shared secret using only its own private key and the sender's session key. A branching node learns that it is a branching node

(because its payload contains multiple `short_channel_ids`), but learns nothing about the total number of remaining hops, the destination's identity, or which branch will ultimately succeed. This is analogous to the limited information leaked in Multi-Path Payments (MPP), where a recipient learns the payment was split — we consider this an acceptable and non-material privacy trade-off.

B. HTLC Atomicity

Node B must prepare HTLCs with both C and D before knowing which will succeed. Sequential locking is preferable for capital efficiency: B first establishes an HTLC with C. If C succeeds, the payment completes and B never contacts D. If C fails within a predefined timeout, B cancels the HTLC with C and establishes one with D. This ensures at most one active HTLC per branch at any time, matching the capital requirement of linear routing. A reasonable timeout default is 2–3 seconds, consistent with existing Lightning HTLC conventions.

C. Onion Truncation and Decryption Integrity

A key concern is whether truncating each successor's onion corrupts XOR-based decryption. The critical invariant is that each node's decryption operation recovers exactly the bytes the source intended. Since the second half of a SPHINX onion is deterministic filler data (not carrying real payloads for downstream hops), a node receiving a truncated half-onion can pad it with zeros before applying its rho-key decryption and correctly recover its payload. This requires that the filler convention be precisely specified in the protocol and that all version-2 nodes implement compatible padding behavior. A formal proof of correctness under the SPHINX security model is identified as future work.

D. Denial-of-Service Mitigations

A malicious source could construct onions with a large branching factor, forcing a node to attempt many sequential HTLC setups. This is mitigated by: (i) capping the maximum supported branching factor per node (suggested maximum $k = 4$); (ii) rate-limiting HTLC setup attempts per incoming payment hash.

VI. IMPACT ON PATH-FINDING

Tesla Routing changes path-finding from finding a single optimal path to finding an optimal directed acyclic subgraph (DAG) from source to destination.

The source identifies a set of paths sharing a common prefix up to the branching node, then diverging into alternative branches.

Algorithms such as Yen's k-shortest-paths can be adapted: the source scores candidate branches by estimated success probability, selects the top k, and encodes them in a single onion. Paths sharing common prefix segments reuse onion construction work, reducing overhead versus constructing k independent onions.

The expected payment success rate under Tesla Routing for $k = 2$ independent branches with success probabilities p_1 and p_2 is:

$$P(\text{success}) = 1 - (1 - p_1)(1 - p_2)$$

For two branches each with $p = 0.70$ (a typical Lightning estimate), $P(\text{success}) = 1 - (0.30)^2 = 0.91$. For three branches at $p = 0.70$, this rises to 0.973. This represents a substantial reliability improvement at the cost of modestly higher fees due to fee equalization.

VII. OPEN PROBLEMS AND FUTURE WORK

A. Formal Security Proof

A rigorous proof that the split-and-interleave construction preserves SPHINX privacy properties — specifically, that no intermediary node gains additional information about path length, destination, or alternative branches — remains an important open problem, likely requiring an extension of the SPHINX security model to accommodate variable-structure payloads.

B. Optimal Branching Factor

As k increases, each branch's onion is more severely truncated, potentially degrading decryption integrity. The optimal k likely depends on path length, network topology, and the distribution of channel success probabilities. Empirical simulation on Lightning Network topology snapshots would help identify practical limits.

C. Interaction with Multi-Path Payments

MPP splits a payment amount across independent paths; Tesla Routing maintains a single amount with redundant routing. Combining both — splitting the amount while providing alternative sub-paths for each split — is a promising direction for maximizing both liquidity utilization and reliability.

D. Incentive Compatibility

Fee equalization overcompensates nodes on cheaper paths. Whether this creates distorted incentives — for instance, encouraging nodes to artificially raise fees to maximize overcompensation — merits game-theoretic analysis. A probability-weighted fee model accounting for the expected selection probability of each branch could address this.

E. Implementation Path

A practical deployment would require updates to BOLT 4 (onion payload format, version 2 specification), BOLT 7 (node_announcement to advertise version support), and major Lightning implementations including LND, CLN, and Eclair. A staged deployment — beginning with testnet experimentation in a small set of nodes — would allow empirical validation before broader adoption.

VIII. RELATED WORK

Pickhardt and Richter [6] proposed a flow-based routing algorithm modeling channel liquidity as a probability distribution to minimize expected payment failures — complementary to Tesla Routing, as their algorithm could select optimal branches to encode in a Tesla onion. Roos et al. [7] studied probing-based approaches to improve liquidity estimation in networks with unknown channel balances. The Trampoline Routing proposal [8] outsources intermediate path-finding to designated nodes; Tesla Routing is fully compatible with trampoline nodes constructing branching onions on behalf of senders. In the anonymity literature, SPHINX [3] and Tor [9] establish the formal security model that this work extends.

IX. CONCLUSION

We have presented Tesla Routing, a protocol extension for the Bitcoin Lightning Network enabling non-linear payment paths. By encoding multiple alternative forwarding branches within a single fixed-size SPHINX onion packet, Tesla Routing allows intermediate nodes to attempt fallback routes without source communication, increasing payment success probability while preserving existing privacy guarantees.

The key contributions are: (1) a split-and-interleave onion construction algorithm encoding k alternative branches within a 1,300-byte payload; (2) a fee

equalization mechanism preserving the single amt_to_forward constraint while fairly compensating fallback nodes; (3) a backward-compatible version negotiation strategy; and (4) analysis of security, privacy, and incentive properties with identification of open problems.

Tesla Routing demonstrates that meaningful reliability improvements are achievable within the existing SPHINX-based packet format constraints, opening a productive direction for research into resilient Lightning Network payment routing.

REFERENCES

- [1] J. Poon and T. Dryja, "The Bitcoin Lightning Network: Scalable Off-Chain Instant Payments," 2016. [Online]. Available: <https://lightning.network/lightning-network-paper.pdf>
- [2] R. Pickhardt and M. Nowostawski, "Imbalance measure and proactive channel rebalancing algorithm for the Lightning Network," in Proc. IEEE Int. Conf. Blockchain and Cryptocurrency, 2019.
- [3] G. Danezis and I. Goldberg, "Sphinx: A compact and provably secure mix format," in Proc. IEEE Symp. Security and Privacy, 2009.
- [4] Lightning Network Specification, "BOLT 4: Onion Routing Protocol." [Online]. Available: <https://github.com/lightning/bolts/blob/master/04-onion-routing.md>
- [5] Lightning Network Specification, "BOLT 7: P2P Node and Channel Discovery." [Online]. Available: <https://github.com/lightning/bolts/blob/master/07-routing-gossip.md>
- [6] R. Pickhardt and S. Richter, "Optimally Reliable and Cheap Payment Flows on the Lightning Network," arXiv:2107.05322, 2021.
- [7] S. Roos et al., "Settling Payments Fast and Private: Efficient Decentralized Routing for Path-Based Transactions," arXiv:1709.05748, 2017.
- [8] J. Jager, "Trampoline Routing," Lightning-dev mailing list, Aug. 2019. [Online]. Available: <https://lists.linuxfoundation.org/pipermail/lightning-dev/2019-August/002100.html>
- [9] R. Dingledine, N. Mathewson, and P. Syverson, "Tor: The Second-Generation Onion Router," in Proc. USENIX Security Symp., 2004.
- [10] A. Antonopoulos et al., Mastering the Lightning Network, Ch. 10: Onion Routing. [Online]. Available:

https://github.com/lnbook/lnbook/blob/develop/10_onion_routing.asciidoc